

CSE 554: Systems for Machine Learning – Assignment 1

Section 1: Implementing the Sigmoid Linear Unit (SiLU)

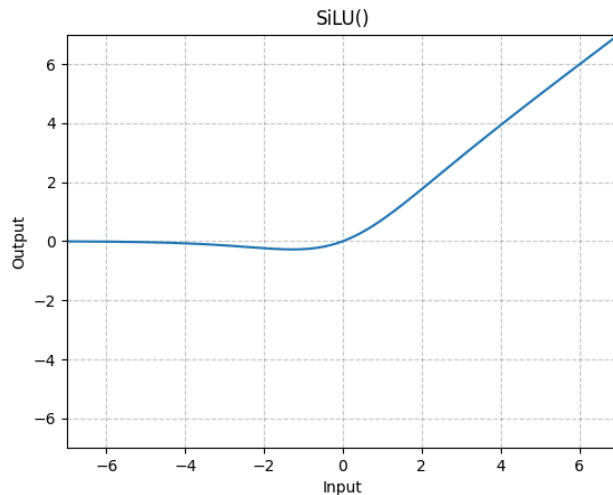


Image source: <https://pytorch.org/docs/stable/generated/torch.nn.SiLU.html>

Q1. 1 + 1 points

- What are the use cases of SiLU in LLMs?

SiLU can be used as an activation function. Compared to ReLU, it has the advantage of being smooth around 0.

SiLU is used in gated FFNs ($\text{SwiGLU}(x) = \text{SiLU}(W_{\text{gate}}x) * W_{\text{up}}x$) because empirically this often achieves better model quality.

- What is the mathematical formula for the SiLU function?

$\text{SiLU}(x) = x * \sigma(x)$ where $\sigma(x) = 1 / (1 + e^{-x})$ is the sigmoid function.

Q2. 2 + 4 + 2 points

- In PyTorch, without using `nn.SiLU` or `nn.Sigmoid`, implement the SiLU function using basic tensor operations for a matrix of shape (8192, 8192) in `silu/silu_torch.py`.
- Use **Torch Profiler** and **NVIDIA Nsight Systems** to profile the execution.
 - Include **screenshots** in the report.Submit the profiling outputs as:

- What is the minimum number (discount repetition and intermediate results) of memory accesses necessary (read and write) to accomplish the task?

The minimum number of memory accesses for silu in terms of bytes is: # bytes of tensor dtype * tensor numel * min # memory accesses for a single tensor element = $4 * (8192 * 8192) * 2 = 536870912$. The minimum number of memory accesses for an element is 2 because we need to read 1 input and write 1 output. The dtype is torch.float32.

- What is the bandwidth usage (minimum number of necessary memory accesses / total execution time) of SILU?

The per-iteration time is 4.2465 ms. The bandwidth usage is 126.43 GB/s.

Q3. 8 + 2 points

- Implement the SiLU function using **Triton** in `silu/silu_triton_kernel.py` and test your kernel in `silu/silu_triton_test.py`.
- Time the custom Triton kernel and report the timing and bandwidth.

The tensor shape is still (8192, 8192). Among block sizes of [64, 128, 256, 512, 1024], 256 performs the best, with a per-iteration time of 0.9480 ms and the bandwidth usage of 566.31 GB/s.

Q4. 8 + 2 + 10 points

- Implement the SiLU function using **CUDA** in `silu/CUDA/silu.cu`. Test your kernel in `silu/CUDA/main.cu`.
- Time the kernel and compute the bandwidth utilization. Use **NVIDIA Nsight Compute (ncu)** to analyze your CUDA kernel. Include the report `cuda_silu.ncu-rep` in your submission.

The per-iteration time is 0.9448 ms and the bandwidth utilization is 566.80 GB/s.

- **Optimize** the kernel to achieve **bandwidth utilization > 500 GB/s**.

Section 2: Implementing RMS Norm

Q1. 1 + 1 + 1 points

- What is the RMS Norm?

RMS Norm stands for root mean square layer normalization and is a normalization that scales a vector by its RMS's inverse as well as a learned per-dimension weight.

- What is RMSNorm used for in LLMs?

RMS Norm is used for normalization in LLMs to stabilize training. Compared to LayerNorm, it involves a little less computation since it does not subtract the mean.

- Provide the formula for RMS Norm.

For $x \in \mathbb{R}^d$, the formula is $\text{RMSNorm}(x) = w * x / \text{RMS}(x)$ where $w \in \mathbb{R}^d$ is the learned

per-dimension weights and $\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$, where the epsilon prevents divisions by zero.

Q2. 10 + 10 points

- Implement a **CUDA kernel** for RMS Norm for a matrix of shape $(8192, 8192)$ in `rms_norm/matrix/rms_norm_matrix.cu`, write a test in `rms_norm/matrix/main.cu`. Time the kernel and compute the bandwidth utilization.

The per-iteration kernel time is 1.35 ms. The minimum number of memory accesses is $8192 * 8192 * 2 + 8192 = 134225920$, and each float32 element is 4 bytes, so the bandwidth usage (as defined earlier) is 397.71 GB/s.

- Optimize it to achieve **bandwidth utilization > 300 GB/s** (attach `rms_norm_matrix.ncu-rep`).

Q3. 10 + 10 points

- Implement a **CUDA kernel** for RMS Norm for a vector of shape $(1, 1024 \times 1024)$ in `rms_norm/vector/rms_norm_vector.cu`, write a test in `rms_norm/vector/main.cu`. Time the kernel and compute the bandwidth utilization.

The per-iteration kernel time is 32.35 μ s. The minimum number of memory accesses is $(1024 * 1024) * 3 = 3145728$, so the bandwidth usage is $3145728 * 4 / (32.35 * 1e-6) / 1e9 = 388.96$ GB/s.

- Optimize this kernel as well to achieve **bandwidth utilization > 200 GB/s** (attach `rms_norm_vector.ncu-rep`).

Q4. 2 points

- Compare the thread block assignment for kernels implemented for $(8192, 8192)$ and $(1, 1024 \times 1024)$. What are the thread block assignment differences, and why were these changes necessary?

For the shape $(8192, 8192)$, the assignment is one block per row. Within a block, the threads perform the reduction to get the sum of squares for that row, then the same

threads compute and write the final RMS Norm for that row. This works because reductions for different rows are independent.

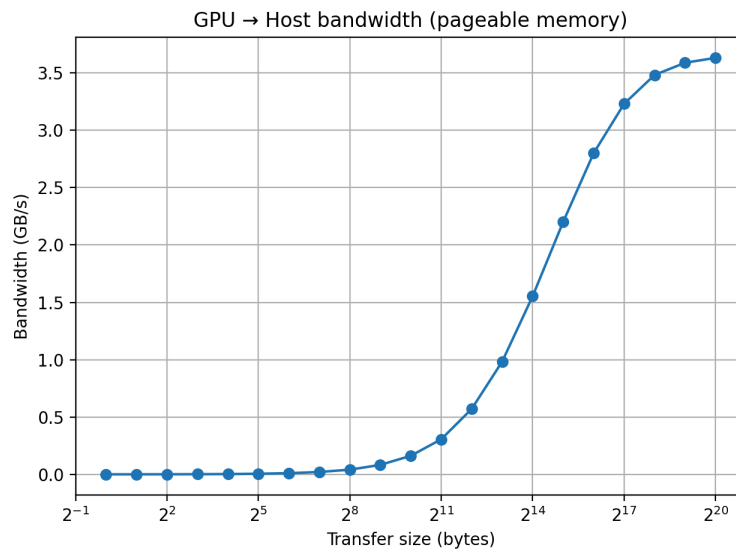
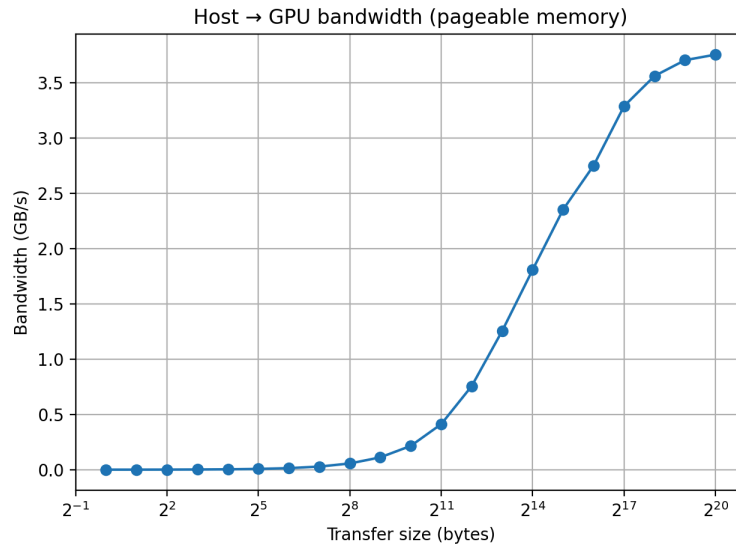
For the shape (1, 1024 * 1024), many blocks cooperate on the same “global” reduction, with each block handling a subset. In this case, we need inter-block synchronization.

The key difference is: one block per row for (8192, 8192) vs. multiple blocks for the only row for (1, 1024 * 1024). The changes are necessary because using 1 block for the vector case would underutilize the GPU.

Section 3: Host–GPU Memory Copy

Q1. 2 + 2 + 1 points

- Measure the memory transfer speed:
 - **Host-to-GPU** and **GPU-to-Host**
For data sizes of 2^0 to 2^{20} bytes for each power of 2.
 - Use **C++**
- Plot a **bandwidth vs. transfer size** curve and include it in your report. Plot two separate curves for **Host-to-GPU** and **GPU-to-Host**.



- What is the **peak transfer speed** observed?

The peak transfer bandwidth observed is 3.76 GB/s when performing host to device transfer of 2^{20} bytes of memory.

Q2. 2 + 2 points

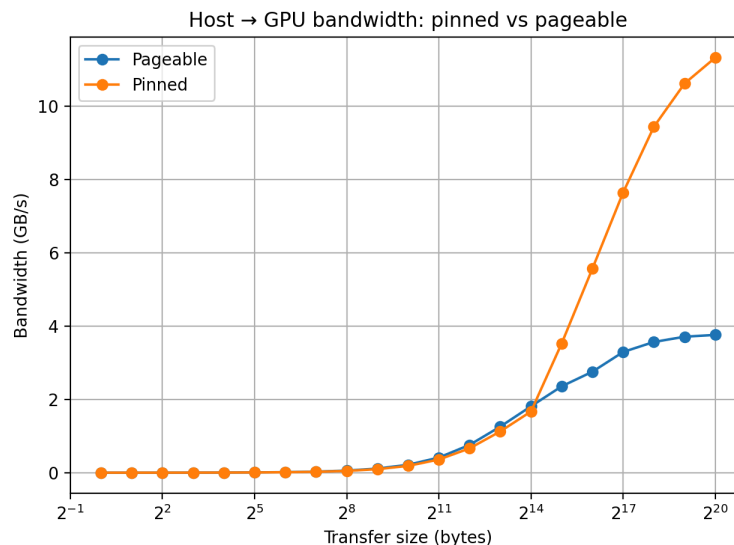
- What is **pinned memory** in the context of NVIDIA GPUs' memory allocations?

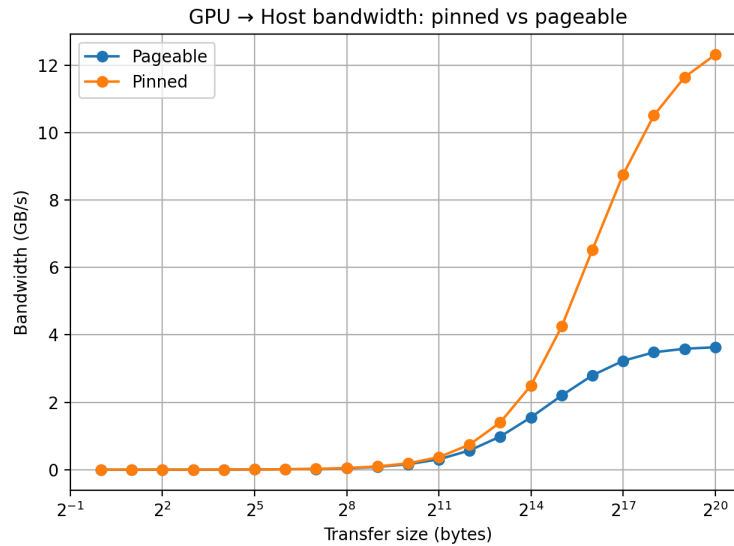
Pinned memory, also called page-locked host memory, is CPU memory that the operating system is not allowed to swap out to disk. It allows the GPU to access directly via DMA (Direct Memory Access).

- How does it impact the performance of memory transfers between the CPU and GPU?
Add a pinned memory curve in the figure for Q1 (Section 3).

Pinned memory improves H2D and D2H copy performance by allowing the GPU to perform direct DMA transfers to and from host memory. In contrast, transfers involving pageable host memory typically require an additional staging step through an internal pinned buffer, introducing extra copying and overhead that reduce effective bandwidth and increase latency.

For small transfer sizes, however, pinned memory provides little benefit because transfer time is dominated by fixed setup and synchronization costs rather than data movement. As the transfer size increases (approximately in the range of 2^{11} to 2^{14} bytes in our measurements) the per-byte transfer cost becomes dominant, and pinned memory achieves higher effective bandwidth by eliminating the need for pageable-memory staging.





Q3. 3 + 3 points

- Assume a matrix (with a dtype of float) of shape `(8192, 65536)` is on the **CPU** in [row-major](#) order. Implement an efficient **C++ host-to-GPU communication** routine in `host_GPU/copy_first_column.cu` to:
 - Copy **only the first column** of each row to the GPU. Assume that the destination buffer on the GPU is a row vector of size `8192`.

The naive implementation copies data one float at a time from pageable host memory using `cudaMemcpy`. The transfer latency is 40.57 ms.

- Optimize** the transfer to ensure the total time is **under 100 μ s** (include `copy.nsys-rep`)

We adopt a two-stage transfer strategy: discontinuous host memory is first packed into a contiguous, pinned host buffer, followed by a `cudaMemcpyAsync` from the pinned buffer to the GPU destination. This approach achieves an end-to-end transfer latency of 82 μ s.